

DOCUMENTACIÓN TÉCNICA DEL ANALIZADOR LÉXICO Y SINTÁCTICO

Este documento describe el diseño, funcionamiento e implementación del analizador léxico y del analizador sintáctico desarrollados para el proyecto del compilador básico en lenguaje C.

El objetivo principal del proyecto es simular las primeras etapas de un compilador real mediante el uso de técnicas de análisis léxico y parsing LR.

1. Analizador Léxico

El analizador léxico fue desarrollado utilizando expresiones regulares de Python mediante el módulo 're'.

Su función principal es leer una cadena de entrada carácter por carácter y dividirla en unidades mínimas llamadas tokens.

El sistema reconoce:

- Identificadores
- Números enteros
- Números reales
- Operadores aritméticos
- Operadores relacionales
- Operadores lógicos
- Palabras reservadas
- Símbolos especiales

1.1 Decisiones de Diseño

Durante el desarrollo del analizador léxico se tomaron las siguientes decisiones importantes:

1. Uso de expresiones regulares:

Permite reconocer patrones de manera eficiente y sencilla.

2. Tabla de símbolos:

Se implementó una estructura de diccionario para relacionar tokens con identificadores numéricos.

3. Prioridad de operadores:

Los operadores compuestos como '==' y '>=' fueron colocados antes que operadores simples para evitar conflictos de reconocimiento.

4. Manejo modular:

El código fue dividido en archivos independientes para facilitar mantenimiento y escalabilidad.

1.2 Manejo de Errores Léxicos

El sistema detecta caracteres inválidos mediante el token ERROR.

Cuando un símbolo no pertenece a la gramática definida, el analizador genera el mensaje:

ERROR LÉXICO: 'símbolo'

Ejemplo:

Entrada:

```
int x = 10 @
```

Salida:

ERROR LÉXICO: '@'

Esto permite identificar rápidamente símbolos no válidos dentro del código fuente.

2. Analizador Sintáctico LR

El analizador sintáctico implementa un parser LR basado en una pila y una tabla de acciones.

El parser recibe como entrada los tokens generados por el analizador léxico y valida si la secuencia cumple las reglas gramaticales definidas.

2.1 Gramáticas Implementadas

Gramática 1:

$E \rightarrow id + id$

Gramática 2:

$E \rightarrow id + E$

$E \rightarrow id$

2.2 Decisiones de Diseño

Las decisiones más importantes durante la implementación fueron:

1. Uso de Parsing LR:

Se eligió el método LR por su capacidad para reconocer gramáticas mediante desplazamientos y reducciones.

2. Uso de una pila:

La pila almacena símbolos y estados para simular el comportamiento de un parser real.

3. Tabla LR:

La tabla define acciones SHIFT, REDUCE y ACCEPT.

4. Conversión simplificada:

Los tokens del léxico son convertidos a símbolos simples como 'id' y '+' antes de ser enviados al parser.

5. Separación modular:

El parser se implementó en un archivo independiente.

2.3 Manejo de Errores Sintácticos

Cuando la tabla LR no contiene una acción válida para el estado y símbolo actual, el sistema genera:

 ERROR SINTÁCTICO

Ejemplo:

Entrada:

id + + id

Resultado:

 ERROR SINTÁCTICO

Esto indica que la secuencia no cumple la gramática definida.

3. Casos de Prueba

3.1 Caso Correcto

Archivo de entrada:

```
int x = 10;  
float y = 3.14;
```

```
if (x >= 10 && y != 0.0) {  
    x = x + 1;  
}
```

Resultado Léxico:

```
int    -> TIPO  
x      -> IDENTIFICADOR  
=      -> =  
10     -> ENTERO  
;      -> ;
```

Resultado Sintáctico:

 CADENA ACEPTADA

3.2 Caso con Error Léxico

Archivo de entrada:

```
int x = 10 @
```

Resultado:

ERROR LÉXICO: '@'

El símbolo '@' no pertenece a la gramática definida.

3.3 Caso con Error Sintáctico

Archivo de entrada:

```
x + + y
```

Resultado:

 ERROR SINTÁCTICO

El parser detecta una secuencia inválida según las reglas LR.

4. Conclusiones

El proyecto permitió implementar de manera práctica las primeras etapas fundamentales de un compilador.

El analizador léxico realiza correctamente el reconocimiento de tokens mediante expresiones regulares, mientras que el parser LR valida estructuras gramaticales utilizando una pila y una tabla de análisis.

La arquitectura modular facilita futuras mejoras como:

- Árboles sintácticos
- Análisis semántico
- Generación de código intermedio
- Optimización

Autor

ESTRADA HUERTA FÉLIX EDUARDO - 216819883